

Raport z projektu: Porządkowanie plików

1. Informacje ogólne

Autor: Katsyaryna Anikevich

Przedmiot: ASU

Temat projektu: Porządkowanie plików

Język programowania: Python

2. Cel projektu

Celem projektu było przygotowanie programu do porządkowania plików w katalogu głównym `X` oraz w dodatkowych katalogach `Y1`, `Y2` itd.

Program wyszukuje problemy w strukturze katalogów i proponuje odpowiednie akcje, takie jak usunięcie plików pustych, usunięcie duplikatów, przeniesienie brakujących plików do katalogu `X`, zmiana nazw plików oraz poprawienie uprawnień.

3. Struktura projektu

```
File-Cleaner/
├─ clean_files.py
├─ create_test_data.sh
├─ create_expected_final_data.sh
├─ README.md
└─ cleaner/
    ├─ actions.py
    ├─ checks.py
    ├─ config.py
    ├─ file_utils.py
    └─ output.py
```

Krótki opis najważniejszych plików:

Plik	Opis
clean_files.py	Główny plik programu. Odpowiada za obsługę argumentów i uruchamianie wybranych sprawdzeń.

Plik	Opis
checks.py	Zawiera funkcje sprawdzające pliki, np. duplikaty, pliki puste, pliki tymczasowe, nowsze wersje i niepoprawne uprawnienia.
file_utils.py	Zawiera funkcje pomocnicze do pracy z plikami, ścieżkami, hashami i uprawnieniami.
config.py	Odczytuje ustawienia z pliku konfiguracyjnego.
actions.py	Obsługuje wybór akcji w trybie interaktywnym.
output.py	Formatuje i wyświetla wyniki działania programu.
create_test_data.sh	Tworzy dane testowe do sprawdzenia działania programu.
create_expected_final_data.sh	Skrypt Bash tworzący oczekiwany wynik końcowy po uruchomieniu programu z opcją <code>--apply</code> .
README.md	Zawiera opis projektu, sposób uruchomienia i strukturę plików.

4. Plik konfiguracyjny

Program korzysta z pliku konfiguracyjnego `.clean_files`.

Przykład konfiguracji:

```
permissions=rw-r--r--

bad_chars=";*?$#`|\

replacement=_

temp_extensions=.tmp,~
```

Konfiguracja określa:

- oczekiwane uprawnienia plików,
- znaki problematyczne w nazwach,
- znak zastępczy dla problematycznych znaków,
- rozszerzenia plików uznawanych za tymczasowe.

5. Sposób uruchomienia programu

Program uruchamia się z linii komend.

Tryb podglądu:

```
python3 clean_files.py --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
```

```
anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: all
Apply changes: False
Interactive mode: False
Files found: 41

=====
EMPTY FILES
=====
1. delete: X/dogs/corgi/empty_corgi_file.txt

=====
TEMPORARY FILES
=====
1. delete: Y1/dogs/labrador/labrador_notes.tmp
2. delete: Y2/rabbits/lop/lop_backup~

=====
NEWER VERSIONS OF FILES
=====
1. replace older: file: X/dogs/dogs_general.txt
   with newer: Y1/dogs/dogs_general.txt
2. replace older: file: X/cats/cat_vaccination.txt
   with newer: Y1/dogs/cat_vaccination.txt
3. replace older: file: X/cats/maine_coon/maine_coon_info.txt
   with newer: Y1/cats/maine_coon/maine_coon_info.txt
```

Tryb automatycznego zastosowania zmian:

```
python3 clean_files.py --apply --config test_data/.clean_files test_data/X test_data/Y1
test_data/Y2
```

```

anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --apply --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: all
Apply changes: True
Interactive mode: False
Files found: 41

=====
EMPTY FILES
=====
1. delete: X/dogs/corgi/empty_corgi_file.txt

=====
TEMPORARY FILES
=====
1. delete: Y1/dogs/labrador/labrador_notes.tmp

2. delete: Y2/rabbits/lop/lop_backup~

=====
NEWER VERSIONS OF FILES
=====
1. replace older: file: X/dogs/dogs_general.txt
   with newer: Y1/dogs/dogs_general.txt

2. replace older: file: X/cats/cat_vaccination.txt
   with newer: Y1/dogs/cat_vaccination.txt

3. replace older: file: X/cats/maine_coon/maine_coon_info.txt
   with newer: Y1/cats/maine_coon/maine_coon_info.txt

```

Tryb interaktywny:

```
python3 clean_files.py --interactive --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
```

```

anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --interactive --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: all
Apply changes: False
Interactive mode: True
Files found: 41

=====
EMPTY FILES
=====
1. delete: X/dogs/corgi/empty_corgi_file.txt

Apply this action? [y]es / [n]o / [a]llways yes / [s]kip all: y

=====
TEMPORARY FILES
=====
1. delete: Y1/dogs/labrador/labrador_notes.tmp

Apply this action? [y]es / [n]o / [a]llways yes / [s]kip all: n
2. delete: Y2/rabbits/lop/lop_backup~

Apply this action? [y]es / [n]o / [a]llways yes / [s]kip all: a

=====
NEWER VERSIONS OF FILES
=====
1. replace older: file: X/dogs/dogs_general.txt
   with newer: Y1/dogs/dogs_general.txt

Apply this action? [y]es / [n]o / [a]llways yes / [s]kip all: █

```

6. Tryby działania programu

Program obsługuje trzy główne tryby:

Tryb	Opis
bez dodatkowych opcji	Program tylko pokazuje znalezione problemy i proponowane akcje.
<code>--apply</code>	Program automatycznie wykonuje wszystkie proponowane akcje.
<code>--interactive</code>	Program pyta użytkownika, czy wykonać daną akcję.

W trybie interaktywnym dostępne są odpowiedzi:

Odpowiedź	Znaczenie
y	wykonaj akcję
n	pomiń akcję
a	zawsze wykonuj akcje tego typu
s	zawsze pomijaj akcje tego typu

7. Wykrywanie plików pustych i tymczasowych

Program wykrywa pliki puste oraz tymczasowe.

Pliki puste to pliki o rozmiarze `0`.

Pliki tymczasowe są rozpoznawane na podstawie rozszerzeń ustawionych w pliku konfiguracyjnym, np. `.tmp` lub `~`.

Dla takich plików program proponuje akcję usunięcia.

```
anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode empty --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: empty
Apply changes: False
Interactive mode: False
Files found: 41

=====
EMPTY FILES
=====
1. delete: X/dogs/corgi/empty_corgi_file.txt

anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode temp --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: temp
Apply changes: False
Interactive mode: False
Files found: 41

=====
TEMPORARY FILES
=====
1. delete: Y1/dogs/labrador/labrador_notes.tmp

2. delete: Y2/rabbits/lop/lop_backup~

anikevich@KatePC:~/File-Cleaner$
```

8. Wykrywanie duplikatów

Program wykrywa duplikaty na podstawie zawartości pliku, a nie tylko nazwy.

Do porównania zawartości wykorzystywany jest hash pliku. Jeżeli kilka plików ma identyczną zawartość, program proponuje pozostawienie najstarszego pliku i usunięcie nowszych kopii.

```
anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode duplicates --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Interactive mode: False
Files found: 41

=====
DUPLICATE FILES
=====
1. delete duplicate: file: Y1/pets_index_backup.txt
   keep oldest: X/pets_index.txt

2. delete duplicate: file: Y2/dogs/corgi/labrador_photo_copy.jpg
   keep oldest: X/dogs/labrador/labrador_photo.jpg

3. delete duplicate: file: Y1/cats/cats_general_copy.txt
   keep oldest: X/cats/cats_general.txt

4. delete duplicate: file: Y1/cats/british/british_copy.txt
   keep oldest: X/cats/british/british_info.txt

5. delete duplicate: file: Y2/cats/maine_coon/another_british_copy.txt
   keep oldest: X/cats/british/british_info.txt

6. delete duplicate: file: Y2/rabbits/rabbits_general_backup.txt
   keep oldest: X/rabbits/rabbits_general.txt

7. delete duplicate: file: Y1/rabbits/lop/lop_info_copy.txt
   keep oldest: X/rabbits/lop/lop_info.txt

8. delete duplicate: file: Y2/hamsters/hamsters_general_copy.txt
   keep oldest: X/hamsters/hamsters_general.txt

9. delete duplicate: file: Y1/hamsters/dwarf/dwarf_info_copy.txt
   keep oldest: X/hamsters/dwarf/dwarf_info.txt
```

9. Wykrywanie nowszych wersji plików

Program sprawdza pliki o tej samej nazwie, ale różnej zawartości.

Jeżeli znajdzie takie pliki, porównuje ich daty modyfikacji i proponuje zastąpienie starszej wersji nowszą.

```
anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode versions --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: versions
Apply changes: False
Interactive mode: False
Files found: 41

=====
NEWER VERSIONS OF FILES
=====
1. replace older: file: X/dogs/dogs_general.txt
   with newer: Y1/dogs/dogs_general.txt

2. replace older: file: X/cats/cat_vaccination.txt
   with newer: Y1/dogs/cat_vaccination.txt

3. replace older: file: X/cats/maine_coon/maine_coon_info.txt
   with newer: Y1/cats/maine_coon/maine_coon_info.txt

4. replace older: file: X/hamsters/syrian/hamster_care.txt
   with newer: Y1/hamsters/syrian/hamster_care.txt

anikevich@KatePC:~/File-Cleaner$
```

10. Przenoszenie brakujących plików do katalogu X

Program sprawdza, czy wszystkie potrzebne pliki znajdują się w katalogu głównym `X`.

Jeżeli plik znajduje się tylko w katalogu `Y1` lub `Y2`, program proponuje przeniesienie go do odpowiedniego miejsca w katalogu `X`.

```
anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2

=====
FILES MISSING IN X
=====
1. move: from: Y1/dogs/cat_vaccination.txt
  to: X/dogs/cat_vaccination.txt

2. move: from: Y1/dogs/dogs_general.txt
  to: X/dogs/dogs_general_1.txt

3. move: from: Y1/dogs/corgi/corgi_training.txt
  to: X/dogs/corgi/corgi_training.txt

4. move: from: Y1/cats/maine_coon/maine_coon_info.txt
  to: X/cats/maine_coon/maine_coon_info_1.txt

5. move: from: Y1/rabbits/angora/angora_care.txt
  to: X/rabbits/angora/angora_care.txt

6. move: from: Y1/hamsters/syrian/hamster_care.txt
  to: X/hamsters/syrian/hamster_care_1.txt

7. move: from: Y2/backup_info.txt
  to: X/backup_info.txt

8. move: from: Y2/dogs/labrador/labrador_health.txt
  to: X/dogs/labrador/labrador_health.txt

9. move: from: Y2/dogs/corgi/corgi_permissions.txt
  to: X/dogs/corgi/corgi_permissions.txt

10. move: from: Y2/cats/british/british_notes.txt
  to: X/cats/british/british_notes.txt
```

11. Poprawianie nazw plików

Program wykrywa pliki, których nazwy zawierają problematyczne znaki, np.:

```
: " ; * ? $ # ` | \
```

Takie znaki są zastępowane znakiem podanym w pliku konfiguracyjnym.

Przykład:

```
plik:test?.txt -> plik_test_.txt
```

```

● anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode names --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: names
Apply changes: False
Interactive mode: False
Files found: 41

=====
FILES WITH PROBLEMATIC NAMES
=====
1. rename: from: X/rabbits/angora/rabbit:care?file#1.txt
   to: X/rabbits/angora/rabbit_care_file_1.txt

```

12. Poprawianie uprawnień plików

Program sprawdza uprawnienia plików.

Jeżeli plik ma inne uprawnienia niż ustawione w konfiguracji, program proponuje ich zmianę.

Przykład oczekiwanych uprawnień:

```
rw-r--r--
```

```

● anikevich@KatePC:~/File-Cleaner$ python3 clean_files.py --mode permissions --config test_data/.clean_files test_data/X test_data/Y1 test_data/Y2
Main directory X: X
Config: test_data/.clean_files
Mode: permissions
Apply changes: False
Interactive mode: False
Files found: 41

=====
FILES WITH INCORRECT PERMISSIONS
=====
1. change permissions: Y2/dogs/corgi/corgi_permissions.txt
   from: rwxrwxrwx
   to:  rw-r--r--

```

13. Wynik działania programu

Po uruchomieniu programu z opcją `--apply` pliki zostają uporządkowane:

- puste i tymczasowe pliki zostają usunięte,
- duplikaty zostają usunięte,
- brakujące pliki zostają przeniesione do katalogu `X`,
- starsze wersje plików zostają zastąpione nowszymi,
- nazwy plików zostają poprawione,
- uprawnienia zostają ujednolicone.

Poprawność działania można sprawdzić przez porównanie katalogu wynikowego z katalogiem `expected_final_data`.

Przykład:

```
diff -r test_data expected_final_data
```

Brak wyniku oznacza, że katalogi są identyczne.

```
expected_final_data/X/rabbits/rabbits_general.txt
● anikevich@KatePC:~/File-Cleaner$ diff -r test_data expected_final_data
○ anikevich@KatePC:~/File-Cleaner$
```

Struktura katalogów przed uruchomieniem programu z opcją `--apply`

```
● anikevich@KatePC:~/File-Cleaner/test_data/X$ tree
```

```
├── cats
│   ├── british
│   │   └── british_info.txt
│   ├── cat_vaccination.txt
│   ├── cats_general.txt
│   └── maine_coon
│       └── maine_coon_info.txt
├── dogs
│   ├── corgi
│   │   ├── corgi_info.txt
│   │   └── empty_corgi_file.txt
│   ├── dogs_general.txt
│   └── labrador
│       ├── labrador_info.txt
│       └── labrador_photo.jpg
├── hamsters
│   ├── dwarf
│   │   └── dwarf_info.txt
│   ├── hamsters_general.txt
│   └── syrian
│       └── hamster_care.txt
├── pets_index.txt
└── rabbits
    ├── angora
    │   ├── angora_info.txt
    │   └── rabbit:care?file#1.txt
    ├── lop
    │   └── lop_info.txt
    └── rabbits_general.txt
```

13 directories, 17 files

Struktura katalogów po uruchomieniu programu z opcją `--apply`

```
anikevich@KatePC:~/File-Cleaner/test_data/X$ tree
.
├── backup_info.txt
├── cats
│   ├── british
│   │   ├── british_info.txt
│   │   └── british_notes.txt
│   ├── cat_vaccination.txt
│   ├── cats_general.txt
│   ├── maine_coon
│   │   └── maine_coon_info.txt
│   └── dogs
│       ├── corgi
│       │   ├── corgi_info.txt
│       │   ├── corgi_permissions.txt
│       │   └── corgi_training.txt
│       ├── dogs_general.txt
│       └── labrador
│           ├── labrador_health.txt
│           ├── labrador_info.txt
│           └── labrador_photo.jpg
├── hamsters
│   ├── dwarf
│   │   ├── dwarf_hamster_facts.txt
│   │   └── dwarf_info.txt
│   ├── hamsters_general.txt
│   └── syrian
│       ├── hamster_care.txt
│       └── syrian_notes.txt
├── pets_index.txt
└── rabbits
    ├── angora
    │   ├── angora_care.txt
    │   ├── angora_extra.txt
    │   ├── angora_info.txt
    │   └── rabbit_care_file_1.txt
    ├── lop
    │   └── lop_info.txt
    └── rabbits_general.txt

13 directories, 25 files
```

14. Podsumowanie

Program spełnia wymagania projektu.

Pozwala wyszukiwać i porządkować pliki w katalogu głównym `X` oraz katalogach kopii `Y1`, `Y2` itd.

Zaimplementowano:

- wykrywanie plików pustych,
- wykrywanie plików tymczasowych,
- wykrywanie duplikatów,
- wykrywanie nowszych wersji plików,
- przenoszenie brakujących plików do katalogu `X`,
- zmianę nazw plików z problematycznymi znakami,
- poprawianie uprawnień,
- tryb podglądu,
- tryb automatycznego wykonywania zmian,

- tryb interaktywny.

Dzięki temu użytkownik może najpierw sprawdzić proponowane działania, a następnie zastosować je automatycznie lub zdecydować o nich w trybie interaktywnym.